Week_2_Assignment

September 7, 2025

1 Week 2 Assignment

[3]:
```
from sklearn import datasets
import numpy as np
import pandas as pd

iris = datasets.load_iris()
```

1.1 Exploration of Data

[27]:
```
iris.data
```

[27]:
```
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2],
       [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3],
       [5. , 3.4, 1.5, 0.2],
       [4.4, 2.9, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.1],
       [5.4, 3.7, 1.5, 0.2],
       [4.8, 3.4, 1.6, 0.2],
       [4.8, 3. , 1.4, 0.1],
       [4.3, 3. , 1.1, 0.1],
       [5.8, 4. , 1.2, 0.2],
       [5.7, 4.4, 1.5, 0.4],
       [5.4, 3.9, 1.3, 0.4],
       [5.1, 3.5, 1.4, 0.3],
       [5.7, 3.8, 1.7, 0.3],
       [5.1, 3.8, 1.5, 0.3],
       [5.4, 3.4, 1.7, 0.2],
       [5.1, 3.7, 1.5, 0.4],
       [4.6, 3.6, 1. , 0.2],
       [5.1, 3.3, 1.7, 0.5],
       [4.8, 3.4, 1.9, 0.2],
```

```
[5. , 3. , 1.6, 0.2],
[5. , 3.4, 1.6, 0.4],
[5.2, 3.5, 1.5, 0.2],
[5.2, 3.4, 1.4, 0.2],
[4.7, 3.2, 1.6, 0.2],
[4.8, 3.1, 1.6, 0.2],
[5.4, 3.4, 1.5, 0.4],
[5.2, 4.1, 1.5, 0.1],
[5.5, 4.2, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.2],
[5. , 3.2, 1.2, 0.2],
[5.5, 3.5, 1.3, 0.2],
[4.9, 3.6, 1.4, 0.1],
[4.4, 3. , 1.3, 0.2],
[5.1, 3.4, 1.5, 0.2],
[5. , 3.5, 1.3, 0.3],
[4.5, 2.3, 1.3, 0.3],
[4.4, 3.2, 1.3, 0.2],
[5. , 3.5, 1.6, 0.6],
[5.1, 3.8, 1.9, 0.4],
[4.8, 3. , 1.4, 0.3],
[5.1, 3.8, 1.6, 0.2],
[4.6, 3.2, 1.4, 0.2],
[5.3, 3.7, 1.5, 0.2],
[5. , 3.3, 1.4, 0.2],
[7. , 3.2, 4.7, 1.4],
[6.4, 3.2, 4.5, 1.5],
[6.9, 3.1, 4.9, 1.5],
[5.5, 2.3, 4. , 1.3],
[6.5, 2.8, 4.6, 1.5],
[5.7, 2.8, 4.5, 1.3],
[6.3, 3.3, 4.7, 1.6],
[4.9, 2.4, 3.3, 1. ],
[6.6, 2.9, 4.6, 1.3],
[5.2, 2.7, 3.9, 1.4],
[5. , 2. , 3.5, 1. ],
[5.9, 3. , 4.2, 1.5],
[6. , 2.2, 4. , 1. ],
[6.1, 2.9, 4.7, 1.4],
[5.6, 2.9, 3.6, 1.3],
[6.7, 3.1, 4.4, 1.4],
[5.6, 3. , 4.5, 1.5],
[5.8, 2.7, 4.1, 1. ],
[6.2, 2.2, 4.5, 1.5],
[5.6, 2.5, 3.9, 1.1],
[5.9, 3.2, 4.8, 1.8],
[6.1, 2.8, 4. , 1.3],
```

```
[6.3, 2.5, 4.9, 1.5],
[6.1, 2.8, 4.7, 1.2],
[6.4, 2.9, 4.3, 1.3],
[6.6, 3. , 4.4, 1.4],
[6.8, 2.8, 4.8, 1.4],
[6.7, 3. , 5. , 1.7],
[6. , 2.9, 4.5, 1.5],
[5.7, 2.6, 3.5, 1. ],
[5.5, 2.4, 3.8, 1.1],
[5.5, 2.4, 3.7, 1. ],
[5.8, 2.7, 3.9, 1.2],
[6. , 2.7, 5.1, 1.6],
[5.4, 3. , 4.5, 1.5],
[6. , 3.4, 4.5, 1.6],
[6.7, 3.1, 4.7, 1.5],
[6.3, 2.3, 4.4, 1.3],
[5.6, 3. , 4.1, 1.3],
[5.5, 2.5, 4. , 1.3],
[5.5, 2.6, 4.4, 1.2],
[6.1, 3. , 4.6, 1.4],
[5.8, 2.6, 4. , 1.2],
[5. , 2.3, 3.3, 1. ],
[5.6, 2.7, 4.2, 1.3],
[5.7, 3. , 4.2, 1.2],
[5.7, 2.9, 4.2, 1.3],
[6.2, 2.9, 4.3, 1.3],
[5.1, 2.5, 3. , 1.1],
[5.7, 2.8, 4.1, 1.3],
[6.3, 3.3, 6. , 2.5],
[5.8, 2.7, 5.1, 1.9],
[7.1, 3. , 5.9, 2.1],
[6.3, 2.9, 5.6, 1.8],
[6.5, 3. , 5.8, 2.2],
[7.6, 3. , 6.6, 2.1],
[4.9, 2.5, 4.5, 1.7],
[7.3, 2.9, 6.3, 1.8],
[6.7, 2.5, 5.8, 1.8],
[7.2, 3.6, 6.1, 2.5],
[6.5, 3.2, 5.1, 2. ],
[6.4, 2.7, 5.3, 1.9],
[6.8, 3. , 5.5, 2.1],
[5.7, 2.5, 5. , 2. ],
[5.8, 2.8, 5.1, 2.4],
[6.4, 3.2, 5.3, 2.3],
[6.5, 3. , 5.5, 1.8],
[7.7, 3.8, 6.7, 2.2],
[7.7, 2.6, 6.9, 2.3],
```

```
       [6. , 2.2, 5. , 1.5],
       [6.9, 3.2, 5.7, 2.3],
       [5.6, 2.8, 4.9, 2. ],
       [7.7, 2.8, 6.7, 2. ],
       [6.3, 2.7, 4.9, 1.8],
       [6.7, 3.3, 5.7, 2.1],
       [7.2, 3.2, 6. , 1.8],
       [6.2, 2.8, 4.8, 1.8],
       [6.1, 3. , 4.9, 1.8],
       [6.4, 2.8, 5.6, 2.1],
       [7.2, 3. , 5.8, 1.6],
       [7.4, 2.8, 6.1, 1.9],
       [7.9, 3.8, 6.4, 2. ],
       [6.4, 2.8, 5.6, 2.2],
       [6.3, 2.8, 5.1, 1.5],
       [6.1, 2.6, 5.6, 1.4],
       [7.7, 3. , 6.1, 2.3],
       [6.3, 3.4, 5.6, 2.4],
       [6.4, 3.1, 5.5, 1.8],
       [6. , 3. , 4.8, 1.8],
       [6.9, 3.1, 5.4, 2.1],
       [6.7, 3.1, 5.6, 2.4],
       [6.9, 3.1, 5.1, 2.3],
       [5.8, 2.7, 5.1, 1.9],
       [6.8, 3.2, 5.9, 2.3],
       [6.7, 3.3, 5.7, 2.5],
       [6.7, 3. , 5.2, 2.3],
       [6.3, 2.5, 5. , 1.9],
       [6.5, 3. , 5.2, 2. ],
       [6.2, 3.4, 5.4, 2.3],
       [5.9, 3. , 5.1, 1.8]])
```

[28]:
```
iris.keys()
```

[28]:
```
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names',
'filename', 'data_module'])
```

[29]:
```
iris['target']
```

[29]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

[32]:
```
iris['target_names']
```

[32]:
```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

[46]:
```
iris['DESCR']
```

[46]:
```
'.. _iris_dataset:\n\nIris plants dataset\n--------------------\n\n**Data Set
Characteristics:**\n\n:Number of Instances: 150 (50 in each of three
classes)\n:Number of Attributes: 4 numeric, predictive attributes and the
class\n:Attribute Information:\n    - sepal length in cm\n    - sepal width in
cm\n    - petal length in cm\n    - petal width in cm\n    - class:\n
- Iris-Setosa\n            - Iris-Versicolour\n            - Iris-
Virginica\n\n:Summary Statistics:\n\n============== ==== ==== ======= =====
====================\n                Min  Max   Mean    SD   Class
Correlation\n============== ==== ==== ======= ===== ====================\nsepal
length:   4.3  7.9   5.84   0.83    0.7826\nsepal width:    2.0  4.4   3.05
0.43   -0.4194\npetal length:   1.0  6.9   3.76   1.76    0.9490  (high!)\npetal
width:    0.1  2.5   1.20   0.76    0.9565  (high!)\n============== ==== ====
======= ===== ====================\n\n:Missing Attribute Values: None\n:Class
Distribution: 33.3% for each of 3 classes.\n:Creator: R.A. Fisher\n:Donor:
Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\n:Date: July, 1988\n\nThe famous
Iris database, first used by Sir R.A. Fisher. The dataset is taken\nfrom
Fisher\'s paper. Note that it\'s the same as in R, but not as in the
UCI\nMachine Learning Repository, which has two wrong data points.\n\nThis is
perhaps the best known database to be found in the\npattern recognition
literature.  Fisher\'s paper is a classic in the field and\nis referenced
frequently to this day.  (See Duda & Hart, for example.)  The\ndata set contains
3 classes of 50 instances each, where each class refers to a\ntype of iris
plant.  One class is linearly separable from the other 2; the\nlatter are NOT
linearly separable from each other.\n\n.. dropdown:: References\n\n  - Fisher,
R.A. "The use of multiple measurements in taxonomic problems"\n    Annual
Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to\n
Mathematical Statistics" (John Wiley, NY, 1950).\n  - Duda, R.O., & Hart, P.E.
(1973) Pattern Classification and Scene Analysis.\n    (Q327.D83) John Wiley &
Sons.  ISBN 0-471-22361-1.  See page 218.\n  - Dasarathy, B.V. (1980) "Nosing
Around the Neighborhood: A New System\n    Structure and Classification Rule for
Recognition in Partially Exposed\n    Environments".  IEEE Transactions on
Pattern Analysis and Machine\n    Intelligence, Vol. PAMI-2, No. 1, 67-71.\n  -
Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule".  IEEE Transactions\n
on Information Theory, May 1972, 431-433.\n  - See also: 1988 MLC Proceedings,
54-64.  Cheeseman et al"s AUTOCLASS II\n    conceptual clustering system finds 3
classes in the data.\n  - Many, many more …\n'
```

[37]:
```
iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)
iris_df['target']=iris.target
iris_df['target_name'] = np.where(iris_df['target']==0,'setosa',np.
 ↪where(iris_df['target']==1,'versicolor','virginica'))
```

[38]:
```
iris_df
```

[38]:
```
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                  5.1               3.5                1.4               0.2
1                  4.9               3.0                1.4               0.2
2                  4.7               3.2                1.3               0.2
3                  4.6               3.1                1.5               0.2
4                  5.0               3.6                1.4               0.2
..                 …                 …                  …                 …
145                6.7               3.0                5.2               2.3
146                6.3               2.5                5.0               1.9
147                6.5               3.0                5.2               2.0
148                6.2               3.4                5.4               2.3
149                5.9               3.0                5.1               1.8

     target target_name
0         0      setosa
1         0      setosa
2         0      setosa
3         0      setosa
4         0      setosa
..      …          …
145       2   virginica
146       2   virginica
147       2   virginica
148       2   virginica
149       2   virginica

[150 rows x 6 columns]
```

[39]:
```
iris_df[35:55]
```

[39]:
```
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
35                5.0               3.2                1.2               0.2
36                5.5               3.5                1.3               0.2
37                4.9               3.6                1.4               0.1
38                4.4               3.0                1.3               0.2
39                5.1               3.4                1.5               0.2
40                5.0               3.5                1.3               0.3
41                4.5               2.3                1.3               0.3
42                4.4               3.2                1.3               0.2
43                5.0               3.5                1.6               0.6
44                5.1               3.8                1.9               0.4
45                4.8               3.0                1.4               0.3
46                5.1               3.8                1.6               0.2
47                4.6               3.2                1.4               0.2
48                5.3               3.7                1.5               0.2
```

```
49                5.0               3.3                1.4               0.2
50                7.0               3.2                4.7               1.4
51                6.4               3.2                4.5               1.5
52                6.9               3.1                4.9               1.5
53                5.5               2.3                4.0               1.3
54                6.5               2.8                4.6               1.5

    target target_name
35       0      setosa
36       0      setosa
37       0      setosa
38       0      setosa
39       0      setosa
40       0      setosa
41       0      setosa
42       0      setosa
43       0      setosa
44       0      setosa
45       0      setosa
46       0      setosa
47       0      setosa
48       0      setosa
49       0      setosa
50       1  versicolor
51       1  versicolor
52       1  versicolor
53       1  versicolor
54       1  versicolor
```

1.1.1 1. How many rows and columns are in iris?

[40]:
```
print(f'The number of rows for the data are {len(iris_df)} and the number of columns are {len(iris_df.columns)}')
```

```
The number of rows for the data are 150 and the number of columns are 6
```

2. Create a new data frame named sub1 which includes only the first 9 rows and the last row of iris. Display sub1.

[41]:
```
sub1=pd.concat([iris_df[0:9],iris_df[-1:]])
```

[42]:
```
sub1
```

[42]:
```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
```

```
4                  5.0               3.6                1.4               0.2
5                  5.4               3.9                1.7               0.4
6                  4.6               3.4                1.4               0.3
7                  5.0               3.4                1.5               0.2
8                  4.4               2.9                1.4               0.2
149                5.9               3.0                5.1               1.8
```

```
     target target_name
0         0      setosa
1         0      setosa
2         0      setosa
3         0      setosa
4         0      setosa
5         0      setosa
6         0      setosa
7         0      setosa
8         0      setosa
149       2   virginica
```

1.1.2 3. Create a new data frame named sub2 which includes only rows in iris where the Sepal.Width is less than 2.4 and includes only the columns Sepal.Length, Sepal.Width, and Species/target. Display sub2.

[44]:
```
iris_df[iris_df['sepal width (cm)']<2.4][['sepal length (cm)','sepal width␣
↪(cm)','target_name']]
```

[44]:
```
     sepal length (cm)  sepal width (cm) target_name
41                 4.5               2.3      setosa
53                 5.5               2.3  versicolor
60                 5.0               2.0  versicolor
62                 6.0               2.2  versicolor
68                 6.2               2.2  versicolor
87                 6.3               2.3  versicolor
93                 5.0               2.3  versicolor
119                6.0               2.2   virginica
```

1.1.3 4. Create a vector named Versicolor_Is_The_Best, which has the value 100 whenever Species/target is equal to "versicolor" and has the value 0 otherwise. Display Versicolor_Is_The_Best.

[45]:
```
Versicolor_Is_The_Best = np.where(iris_df['target_name']=='versicolor',100,0)
Versicolor_Is_The_Best
```

[45]:
```
array([  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
         0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
         0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
         0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0, 100, 100,
```

```
100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
100, 100, 100, 100, 100, 100, 100, 100, 100,   0,   0,   0,   0,
  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
  0,   0,   0,   0,   0,   0,   0])
```

1.1.4 5. Save the column named Sepal.Width as its own vector named sw. Use functions to find the mean, median, maximum, and minimum of sw.

[50]:
```
sw = iris_df['sepal width (cm)']

print(f'Analysis of the Sepal With: \n - The mean of the distribution is {np.
 ↪mean(sw)}.  \n - The median of the distribution is {np.percentile(sw,50)} \n␣
 ↪- The maximum of the distribution is {np.max(sw)} \n - The minimum of the␣
 ↪distribution is {np.min(sw)} ')
```

```
Analysis of the Sepal With:
 - The mean of the distribution is 3.0573333333333337.
 - The median of the distribution is 3.0
 - The maximum of the distribution is 4.4
 - The minimum of the distribution is 2.0
```

1.1.5 6. Use a loop to add up the values in sw one at a time until the sum first exceeds 100. What is this sum, and how many times did the loop have to execute to reach it?

[For example, if we were adding up the values in sw until the sum first exceeded 10 instead, the answer would be 3.5+3.0+3.2+3.1 = 12.8 after 4 loops. Also, be aware this could be done more efficiently in R without using loops, but I want you to get the practice.]

[57]:
```
x = 0
for i in range(len(sw)):
    x += sw[i]
    if x>100:
        print(f'The sum of the first {i} values oif the Sepal With exceeds 100,␣
 ↪the sum is equal to {x}')
        break
```

```
The sum of the first 28 values oif the Sepal With exceeds 100, the sum is equal
to 100.30000000000001
```

1.1.6 7. Create a new function called cmtoin() that converts centimeters to inches (1 inch = 2.54 cm). The values in sw are currently recorded in centimeters. Apply your function to sw and save the result as a new vector named sw_in. Display the first 7 values of sw_in.

[59]:
```
def cmtoin(x: pd.core.series.Series):
    return x/2.54
```

[63]:
```
sw_in=cmtoin(sw)
```

[64]:
```
sw_in[:7]
```

[64]:
```
0    1.377953
1    1.181102
2    1.259843
3    1.220472
4    1.417323
5    1.535433
6    1.338583
Name: sepal width (cm), dtype: float64
```

1.1.7 8. Create a plot that compares Sepal.Length to Petal.Length. Add some informative/interesting axis labels, colors, etc. Have some fun with it.

[Try running ?plot in R for some guidance.]

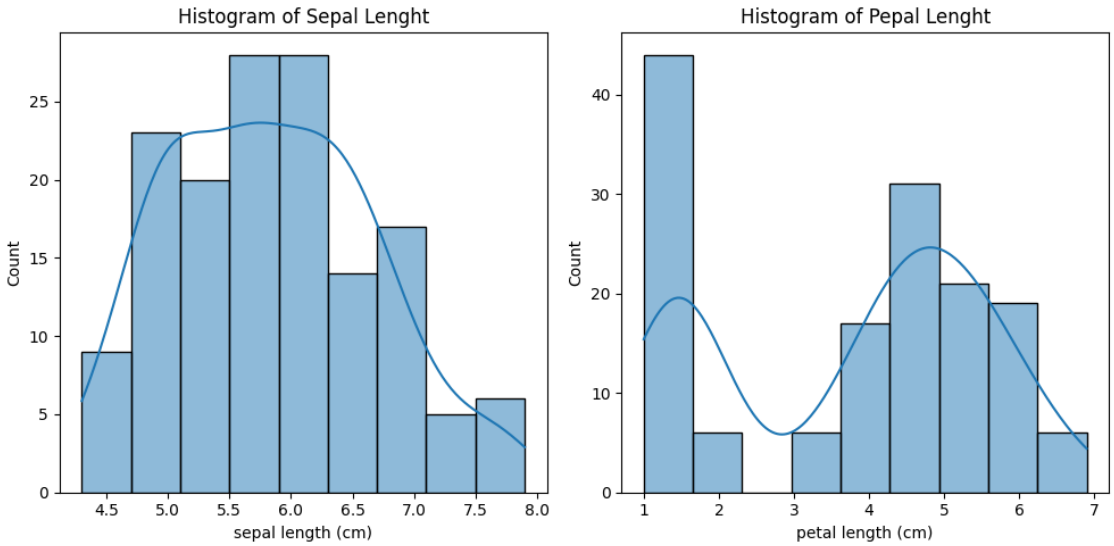
[86]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Create a figure with a 1x2 grid of subplots
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# Plot on the first subplot (axes[0])
sns.histplot(x="sepal length (cm)", data=iris_df, ax=axes[0], kde=True)
axes[0].set_title('Histogram of Sepal Lenght')

# Plot on the second subplot (axes[1])
sns.histplot(x="petal length (cm)", data=iris_df, ax=axes[1], kde=True)
axes[1].set_title('Histogram of Pepal Lenght')

plt.tight_layout() # Adjust layout to prevent overlapping
plt.show()
```



Now we can analyse the distribution betweeen the different species

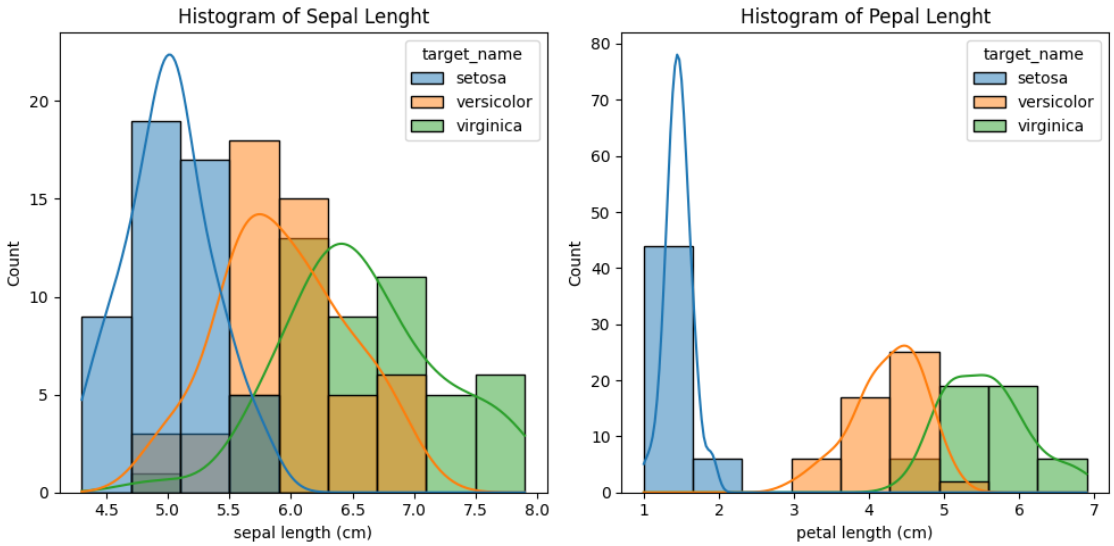
[87]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Create a figure with a 1x2 grid of subplots
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# Plot on the first subplot (axes[0])
sns.histplot(x="sepal length (cm)", data=iris_df, ax=axes[0],␣
 ↪hue='target_name', kde=True)
axes[0].set_title('Histogram of Sepal Lenght')

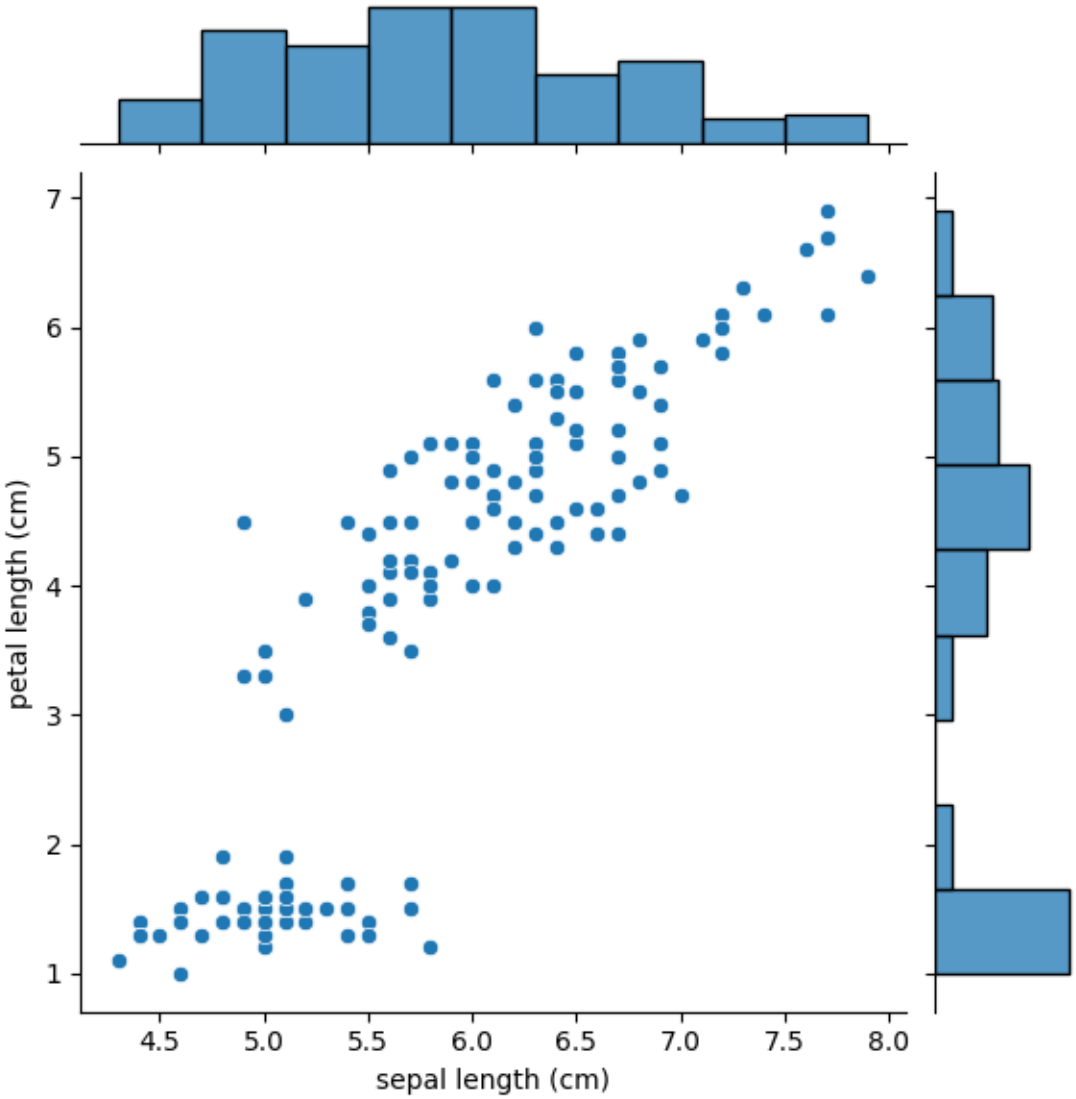
# Plot on the second subplot (axes[1])
sns.histplot(x="petal length (cm)", data=iris_df, ax=axes[1],␣
 ↪hue='target_name', kde=True)
axes[1].set_title('Histogram of Pepal Lenght')

plt.tight_layout() # Adjust layout to prevent overlapping
plt.show()
```

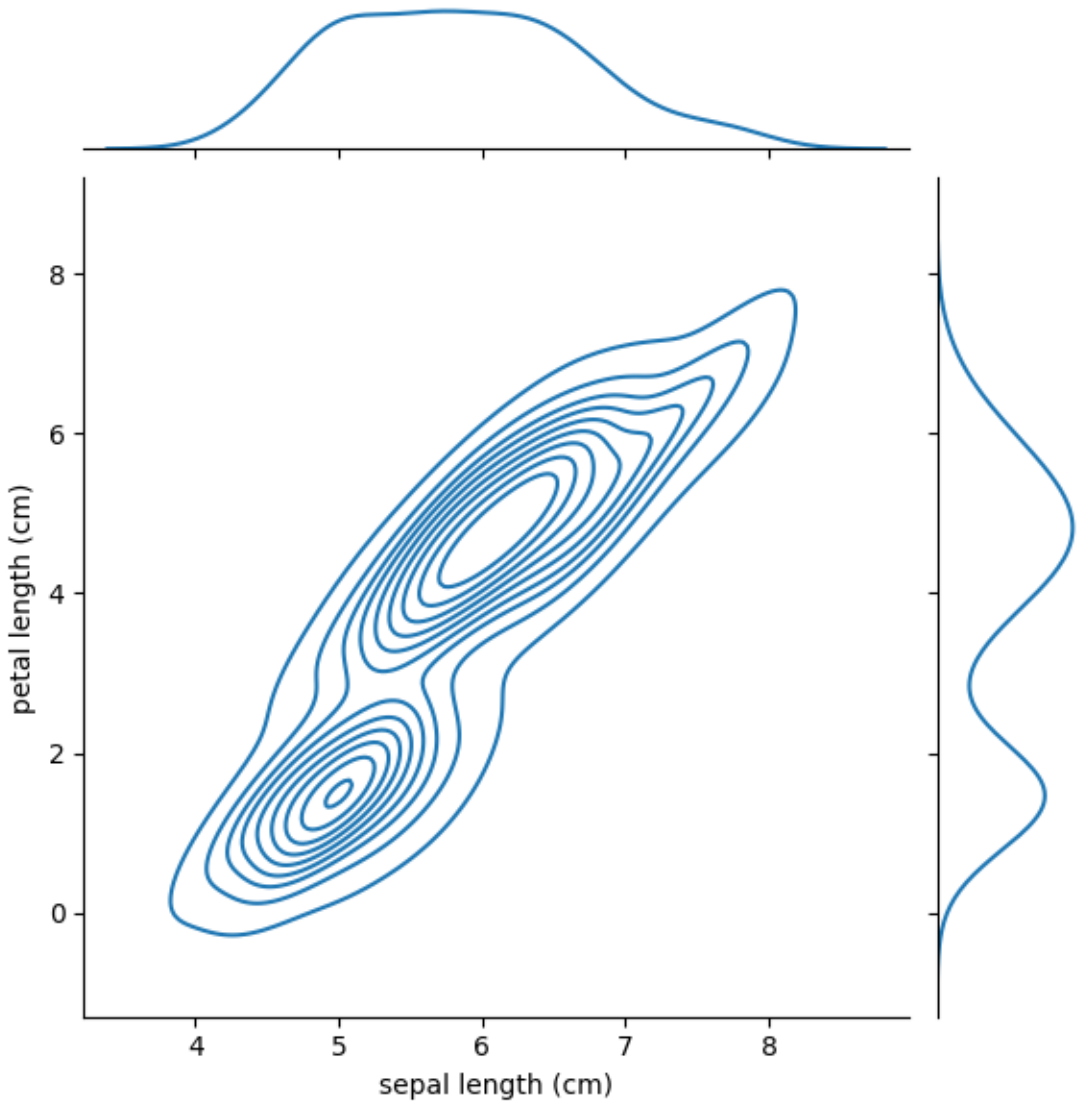


Now we can analyse the relationship between the two variables

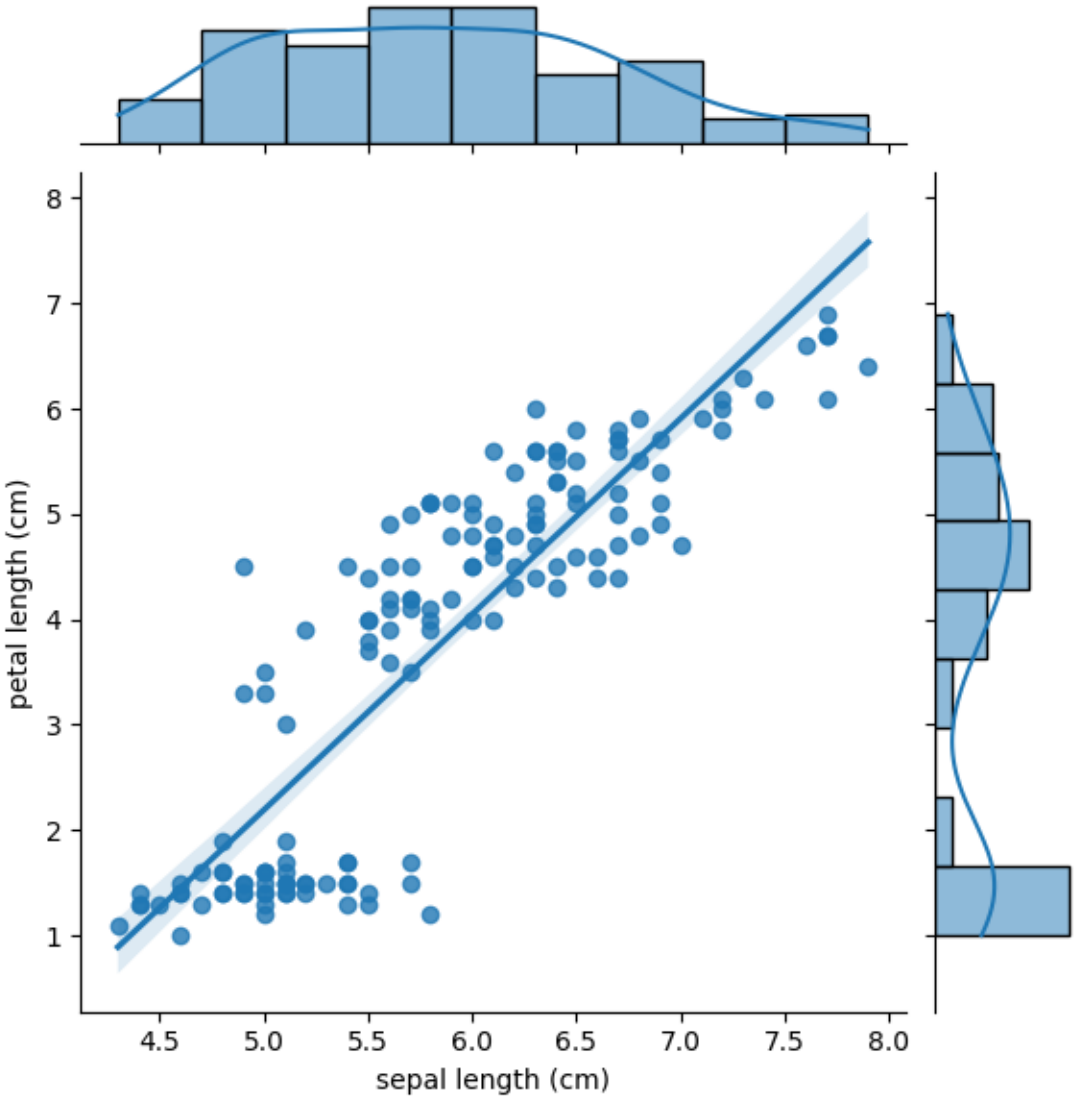
[70]:
```
sns.jointplot(x="sepal length (cm)", y="petal length (cm)", data=iris_df,␣
↪kind="scatter") # or "scatter", "reg", etc.
plt.show()
```



[74]:
```
sns.jointplot(x="sepal length (cm)", y="petal length (cm)", data=iris_df,␣
↪kind="kde") # or "scatter", "reg", etc.
plt.show()
```

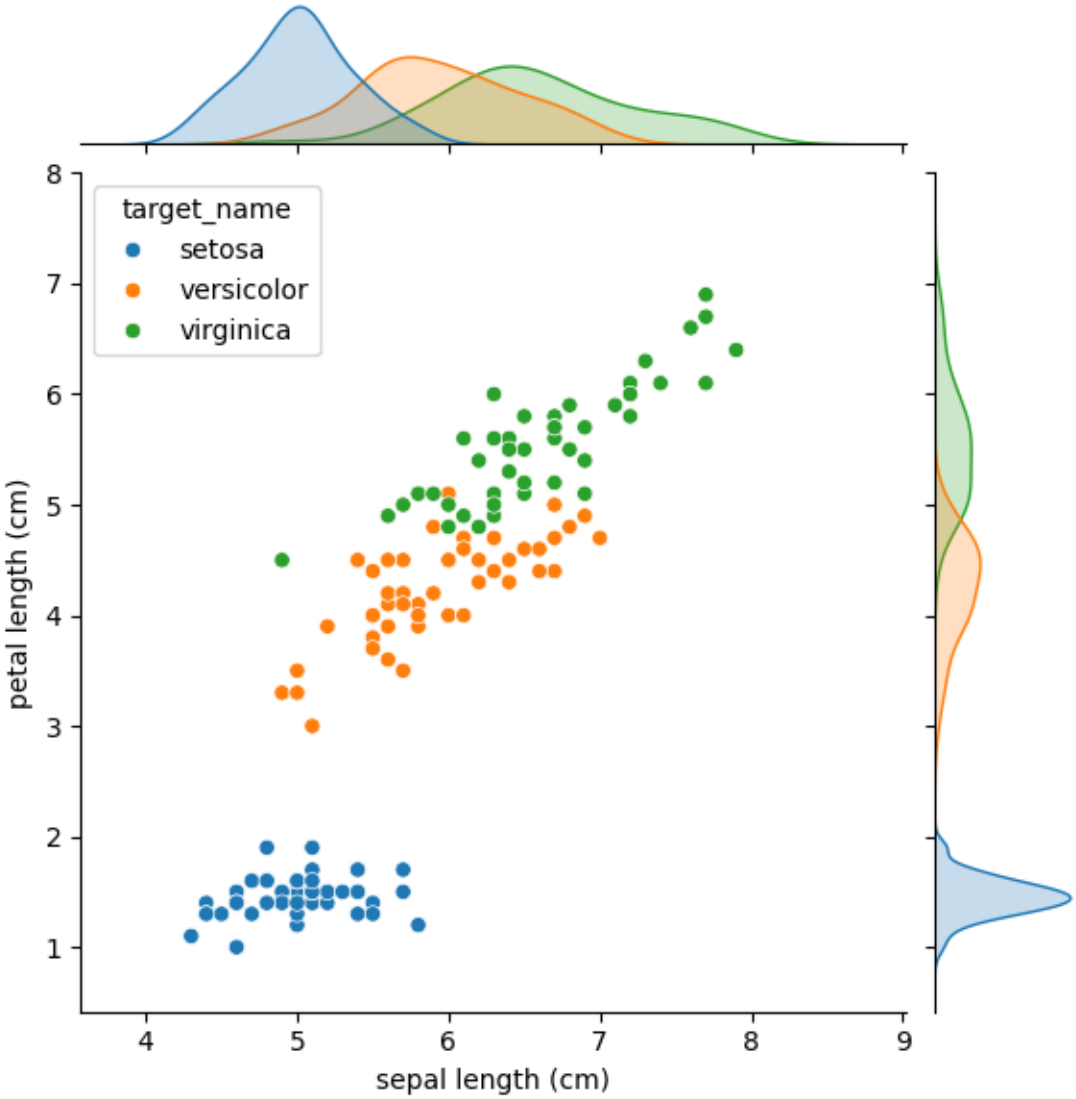


[75]:
```
sns.jointplot(x="sepal length (cm)", y="petal length (cm)", data=iris_df,␣
↪kind="reg") # or "scatter", "reg", etc.
plt.show()
```

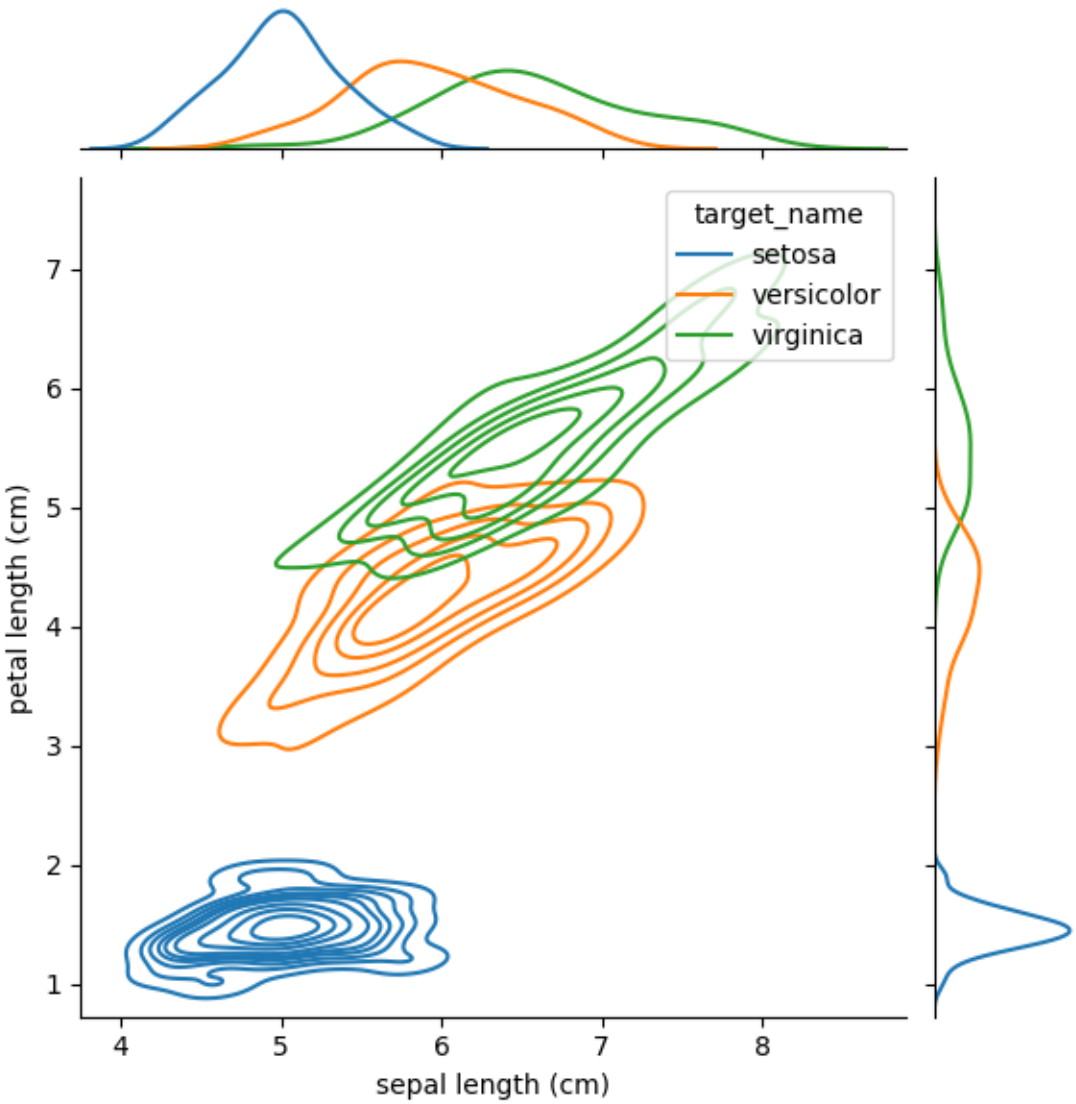


Now we can see the relationship of the two variables for each of the species

[76]:
```
sns.jointplot(x="sepal length (cm)", y="petal length (cm)",␣
↪hue='target_name',data=iris_df, kind="scatter") # or "scatter", "reg", etc.
plt.show()
```



[78]:
```
sns.jointplot(x="sepal length (cm)", y="petal length (cm)",␣
↪hue='target_name',data=iris_df, kind="kde") # or "scatter", "reg", etc.
plt.show()
```


target_name
setosa
versicolor
virginica
petal length (cm)
sepal length (cm)
1
2
3
4
5
6
7
4
5
6
7
8